



Tiffin

Complete JavaScript Tutorial & Code Explanation

A Full-Stack Web App for Nutrition Tracking

Created: September 2026

For: Complete Beginners to Advanced Developers

Table of Contents

- Module 1: JavaScript Foundations
- Module 2: The Document Object Model (DOM)
- Module 3: Events & Interactivity
- Module 4: State Management
- Module 5: Data Structures
- Module 6: The Tiffin App Architecture
- Module 7: Deep Dive: Core Functions
- Module 8: UI Rendering System
- Appendix: Complete Glossary

Module 1: JavaScript Foundations

1.1 What is JavaScript?

JavaScript is a programming language that runs in web browsers. Unlike HTML (structure) and CSS (styling), JavaScript makes web pages interactive and dynamic. In Tiffin, JavaScript handles all the logic: storing data, calculating nutrients, and updating the display.

1.2 Variables: Storing Data

Variables are containers for storing information. Think of them as labeled boxes—each box holds a value.

const - A constant (cannot change):

```
const name = 'Alice';  
const age = 25;
```

let - A variable (can change):

```
let score = 100;  
score = 150;
```

1.3 Data Types

Number: 123, 45.67, -10

String: 'hello' or "world"

Boolean: true or false

Array: [1, 2, 3, 'apple']

Object: {name: 'Alice', age: 25}

null: Intentional absence of value

1.4 Operations & Operators

Addition: $5 + 3 = 8$

Comparison: $5 > 3$ is true

String concatenation: 'Hello' + ' ' + 'World'

Logical AND: true && false = false

Logical OR: true || false = true

Module 2: The Document Object Model (DOM)

2.1 What is the DOM?

The DOM is a tree of all HTML elements on a page. JavaScript uses the DOM to read and modify HTML. When you update the DOM, the browser automatically re-renders the page.

2.2 Selecting Elements

Get an element by ID:

```
const btn = document.getElementById('myButton');
```

Get elements by class:

```
const items = document.querySelectorAll('.food-item');
```

2.3 Modifying HTML

Change text:

```
element.textContent = 'New text';
```

Change HTML:

```
element.innerHTML = '<b>Bold</b>';
```

Add a CSS class:

```
element.classList.add('active');
```

Module 3: Events & Interactivity

3.1 What are Events?

Events are things that happen on a page: clicks, typing, scrolling. JavaScript can listen for events and run code in response.

3.2 Listening for Events

```
button.addEventListener('click', function() {  
  console.log('Button clicked!');  
});
```

3.3 Common Events

- **click:** Triggered when user click
- **change:** Triggered when user change
- **submit:** Triggered when user submit
- **keydown:** Triggered when user keydown
- **mouseover:** Triggered when user mouseover
- **scroll:** Triggered when user scroll

Module 4: State Management

4.1 What is Application State?

State is the current data of your app. In Tiffin, state includes: user profile, nutrition goals, meals logged, weight history, etc. State changes when users interact with the app.

4.2 The State Object in Tiffin

```
let STATE = {  
  goals: { calorieGoal: 2000, proteinGoal: 100, ... },  
  profile: { name: '', sex: null, age: null, ... },  
  diary: { '2024-09-17': { 'meal-1': [...], ... } },  
  customFoods: [...],  
  weightLog: [...],  
};
```

4.3 localStorage: Persistent Storage

localStorage is browser storage that persists across sessions. Tiffin saves state to localStorage so data survives page refreshes.

```
// Save state  
localStorage.setItem('tiffin_state_v1', JSON.stringify(STATE));  
  
// Load state  
const saved = localStorage.getItem('tiffin_state_v1');  
STATE = JSON.parse(saved);
```

Module 5: Data Structures

5.1 Understanding Nutrients

Tiffin tracks 44 nutrients: macronutrients (protein, carbs, fat), micronutrients (vitamins, minerals), and more. Each food has a nutrient object:

```
{
  kcal: 100,
  protein: 10,
  carbs: 5,
  fat: 2,
  fiber: 1,
  ...
}
```

5.2 Food Items Structure

```
{
  id: 'food-123',
  name: 'Chicken Breast',
  servingLabel: '100g',
  servingGrams: 100,
  category: 'Proteins',
  kcal: 165, protein: 31, carbs: 0, fat: 3.6,
  ...(all 44 nutrients)
}
```

5.3 Diary Structure (Daily Log)

The diary stores meals for each day. Date keys are strings like '2024-09-17':

```
diary: {
  '2024-09-17': {
    'meal-1': [
      { ref: {type: 'db', id: 'food-123'}, qty: 1.5 },
    ],
    water: 2000,
  }
}
```

Module 6: The Tiffin App Architecture

6.1 Overall Flow

1. **User Opens App:** app.js loads, STATE is loaded from localStorage
2. **Render Phase:** JavaScript generates HTML for the current view
3. **User Interacts:** Click, type, submit—events trigger handler functions
4. **Update Phase:** Handler modifies STATE
5. **Save & Re-render:** STATE is saved to localStorage, page updates

6.2 Main Components

Food Database (FOOD_DB): Pre-loaded JSON with 3000+ foods and nutrients

State (STATE): Current app data: profile, goals, diary, etc.

Rendering Functions: Generate HTML (renderToday, renderFoodsView, etc.)

Handler Functions: Respond to user events (addFood, logMeal, updateProfile, etc.)

Utility Functions: Helpers: calculate nutrients, format dates, etc.

6.3 Key Views

Home/Today: Log meals for today, see daily summary

Foods: Search/browse foods, create custom foods

Progress: Track weight, view charts, see streaks

Nutrients: See detailed breakdown of all nutrients

Profile: Set personal info, goals, targets

Module 7: Deep Dive—Core Functions

7.1 Utility Functions

uid(): Generate unique IDs

```
const id = uid(); // Returns 'a3c2f1x'
```

toKey(date): Convert Date to string key

```
const key = toKey(new Date()); // Returns '2024-09-17'
```

zeroNutrients(): Create empty nutrient object

```
const empty = zeroNutrients(); // All nutrients = 0
```

7.2 Nutrient Math

scaleNutrients(base, qty): Multiply nutrients by quantity

```
// If eating 1.5 servings of rice
const nutrients = scaleNutrients(riceNutrients, 1.5);
```

addNutrients(a, b): Sum two nutrient objects

```
// Total daily nutrients
const total = addNutrients(mealTotals, snackTotals);
```

7.3 Diary Functions

getDay(dateKey): Get or create diary entry for a date

```
const day = getDay('2024-09-17');
```

dayTotals(dateKey): Sum all nutrients for a day

```
const total = dayTotals('2024-09-17');
```

mealTotal(dateKey, mealId): Sum nutrients for one meal

```
const mealNutrients = mealTotal('2024-09-17', 'meal-1');
```

7.4 Energy Calculations

calcBMR(): Basal Metabolic Rate (calories at rest)

calcTDEE(): Total Daily Energy Expenditure (with activity multiplier)

calculateTargetsFromProfile(): Auto-calculate macro goals based on profile and goals

Module 8: The UI Rendering System

8.1 Render Functions

Rendering functions generate HTML as strings. They take data and return HTML. The main render function updates the page:

```
function renderApp() {  
  renderShell(); // Navigation, sidebar  
  renderView(); // Current view (today, foods, etc.)  
}
```

8.2 View Rendering Functions

renderToday(): Home view—meals, daily totals

renderFoodsView(): Food search and custom food creation

renderProgressView(): Weight tracking and charts

renderNutrientsView(): Detailed nutrient breakdown

renderProfileView(): User profile and goal settings

8.3 Building HTML with Data

```
function renderFoodItem(food) {  
  return `  
    <strong>${escapeHtml(food.name)}</strong>  
    <span>${food.kcal} kcal</span>  
  </div>`;  
}  
  
// Render many items  
const html = foods.map(f => renderFoodItem(f)).join('');  
document.getElementById('foodList').innerHTML = html;
```

8.4 The Render Cycle

Step 1: User clicks button, event handler fires

Step 2: Handler modifies STATE

Step 3: Handler calls `saveState()` (localStorage update)

Step 4: Handler calls `renderView()` or `renderApp()`

Step 5: New HTML replaces old HTML in DOM

Step 6: Browser re-renders the page (user sees update)

8.5 Event Handlers

Event handlers respond to user actions. They live in main.html <script> tag. Example:

```
document.addEventListener('click', function(e) {  
  if(e.target.dataset.action === 'add-food') {  
    const foodId = e.target.dataset.foodId;  
    const qty = parseFloat(e.target.dataset.qty);  
    const mealId = e.target.dataset.mealId;  
    addFoodToMeal(foodId, mealId, qty);  
  }  
});
```

8.6 Complete Example: Adding a Food

User clicks 'Add Food' button for Chicken (1 serving)

1. Event handler reads food ID and meal ID
2. Creates entry: `{ref: {type: 'db', id: 'chicken'}, qty: 1}`
3. Adds entry to `STATE.diary['2024-09-17']['meal-1']`
4. Calls `saveState()` → saves to `localStorage`
5. Calls `renderView()` → re-renders meals
6. User sees chicken added to today's log

Appendix: Glossary

API: Application Programming Interface—set of functions/rules for code interaction

Array: Ordered collection of items: [1, 2, 3]

Boolean: true or false

Callback: Function passed to another function, called later

DOM: Document Object Model—tree of HTML elements

Event: Something that happens: click, type, scroll

Function: Reusable block of code

JSON: JavaScript Object Notation—text format for data

Object: Collection of key-value pairs: {name: 'Alice', age: 25}

Render: Convert data to HTML and update the page

State: Current data of the application

String: Text: 'hello' or "world"

Template Literals: Strings with variables: `Hello \${name}`

const: Constant—variable that cannot be reassigned

localStorage: Browser storage for persistent data

Created: September 17, 2026

App: Tiffin - Nutrition Tracker

Technology: Vanilla JavaScript, HTML5, CSS3, localStorage